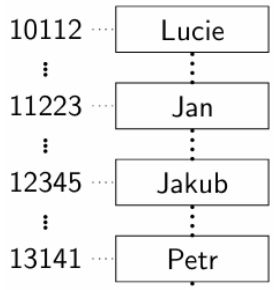
Rozptylová tabulka

- Vlastnosti ADT Tabulka:
  - klíče jsou unikátní
  - není určeno pořadí prvků
- **Tabulka realizovaná seznamem**
  - Pro realizaci dvojic použijeme tuple
  - data = [("T12345P", "Jakub"), ("T45678P", "Anna"), ("T78901P", "Martin")]
  - Hledání v neseřazené: $\Theta(n)$
  - Seřazená
    -
```python
data = ...

data.sort(key=lambda tup: tup[0]) # Seřadíme data podle prvního prvku

code = "T78901P"                  # Zvolíme data, která chceme najít

# Zavoláme funkci bisect_left pro nalezení indexu kódu
index = bisect.bisect_left(data, (code,))

# Zkontrolujeme výsledek vyhledávání
if index != len(data) and data[index][0] == code:
    print(f"Jméno odpovídající {code} je {data[index][1]}.")
else:
    print(f"{code} není v poli dat.")
```
    - Hledání v seřazené $\Theta(\log_2(n))$
- **Tabulka realizovaná dvěma seznamy**
  - Každému klíči na indexu i z pole klíčů odpovídají data na stejném indexu i z pole hodnot
    - codes = ["T12345P", "T45678P", "T78901P"]
    - names = ["Jakub", "Anna", "Martin"]
  - Složitosti stejné ale menší nárok na cash
- **Tabulka s přímým adresováním**
  - Máme jen pole hodnot.
  - Index do pole spočítáme přímo z klíče
    - 

```python
data = [("T12345P", "Jakub"), ("T45678P", "Anna"), ("T78901P", "Martin"),
↪  ("T10112P", "Lucie"), ("T11223P", "Jan"), ("T13141P", "Petr")]

new_data = [None] * 100000        # Vytvoříme nové pole s číselnými klíči
for (key, name) in data:
    key = key.replace("T", "").replace("P", "") # Odstraníme písmena z klíče
    key = int(key)                              # Převedeme klíč na celé číslo
    new_data[key] = name          # Přidáme jméno na index odpovídající klíči

key = T78901P                     # Zvolíme klíč, který chceme najít
key = key.replace("T", "").replace("P", "") # Odstraníme písmena z klíče

# Zkontrolujeme výsledek vyhledávání
if key < len(new_data) and new_data[key] is not None:
    print(f"Jméno odpovídající {key} je {new_data[key]}.")
else:
    print(f"{key} není v poli dat.")
```
  - Složitost hledání: $\Theta(1)$

- **Rozptylová tabulka (Hash table)**
  - cíl
    - rovnoměrné zaplnění
    - rychlé vyhledávání
  - Implementace
    - Obdobná jako Tabulka s přímým adresováním.
    - Kolize mohou být řešení dalšími vnořenými seznamy.
  - Zaplnění tabulky (load factor)
    - průměrný počet položek na přihrádku
    - $$\lambda = \frac{\text{počet položek } n}{\text{počet přihrádek } m}$$
      - velké λ → hodně kolizí → zpomalení operací
      -
      - Malé λ → hodně prázdných položek → nevyužitá paměť
  - Rozptylová funkce
    - Náhodné hodnoty – rovnoměrné zaplňování seznamu přihrádek.
    - Pro různé klíče by měla dávat různé hodnoty
      - nelze vždy, protože počet možných klíčů může být významně menší než délka pole
    - Pro stejné hodnoty musí dávat stejné hodnoty
      - normalizace klíče
    - Rovnost hodnoty rozptylové funkce pro různé klíče: <u>kolize</u>
    - V pythonu vestavěná hash()
      -
      - Pro vlastní objekty nutné implementovat __eq__ a __hash__
        -
```python
hash(123)                          # 123
hash(123456789123456789123456789) # 1361126810988140292

hash("python")                     # -5859315622727106325

hash(("Petr", "p@a.cz"))           # 2812264813959488621

hash({1, 2, 3})                    # TypeError: unhashable type: 'set'
hash(frozenset({1, 2, 3}))         # -2723754012242171 60
```

```python
    def __eq__(self, value: object) -> bool:
        if not isinstance(value, User):
            return False

        return self.name == value.name and self.email == value.email

    def __hash__(self) -> int:
        return hash((self.name, self.email))

hash(User("Petr","p@a.cz"))                    # 2061414140790974 67
hash(User("Petr","p@a.cz"))                    # 2061414140790974 67
User("Petr","p@a.cz") == User("Petr","p@a.cz") # True
```
  - Složitosti
    - Očekávané (průměrné):
      - Přidání přiřazení (klíč + hodnota): Θ(1).
      - Získání hodnoty pro daný klíč: Θ(1).
      - Zjištění, zda je klíči přiřazena hodnota: Θ(1).
      - Zrušení přiřazení (odebrání klíče): Θ(1).
    - Nejhorší případ:
      - všechny klíče se „shodnou“ (kolidují) → všechny skončí na stejném indexu → tabulka musí projít až n položek → Θ(n).
        - Přidání přiřazení (klíč + hodnota): Θ(n).
        - Získání hodnoty pro daný klíč: Θ(n).
        - Zjištění, zda je klíči přiřazena hodnota: Θ(n).
        - Zrušení přiřazení (odebrání klíče): Θ(n).